

```

commit 543c5a825f3ccad295b1c88f2a4abce00c8791 (HEAD -> main)
Author: Anchal Agarwal
Date: Wed Jan 17 08:55:23 2024 +0530

    Adding third commit

commit bfb79d72518aebdd5e7df7425bdcefc16e62552
Author: Anchal Agarwal
Date: Wed Jan 17 08:13:03 2024 +0530

    Adding second commit

commit ac6a9797c263b7582ef2afb6dca81a4934f5c89c
Author: Anchal Agarwal
Date: Mon Jan 15 23:37:42 2024 +0530

    Adding first commit
(END)

```

Figure 9: Output of Git log before the *git cherry-pick* command is given

```

bfb79d (HEAD -> main) HEAD@{0}: reset: moving to bfb79d72518aebdd5e7df7425bdcefc16e62552
ba3fe0a HEAD@{1}: cherry-pick: Adding fifth commit
543c5a8 HEAD@{2}: checkout: moving from branch2 to main
2ff79a9 (branch2) HEAD@{3}: checkout: moving from main to branch2
543c5a8 HEAD@{4}: checkout: moving from branch2 to main
2ff79a9 (branch2) HEAD@{5}: checkout: moving from main to branch2
543c5a8 HEAD@{6}: checkout: moving from branch2 to main
2ff79a9 (branch2) HEAD@{7}: checkout: moving from main to branch2
543c5a8 HEAD@{8}: checkout: moving from branch2 to main
2ff79a9 (branch2) HEAD@{9}: commit: Adding fifth commit
1793a1a HEAD@{10}: commit: Adding fourth commit
bfb79d (HEAD -> main) HEAD@{11}: checkout: moving from main to branch2
543c5a8 HEAD@{12}: checkout: moving from branch2 to main

```

Figure 10: Running the *git reflog* command

This is what the output will look like after the *git cherry-pick* command is executed in the main branch:

```

→ git_tutorial_dir git:(branch2) git checkout main
Switched to branch 'main'
→ git_tutorial_dir git:(main) cat file2.txt
I am creating a file2
→ git_tutorial_dir git:(main) git cherry-pick
2ff79a9ae0bf13a0114ce3056dc114b12053dcb
[main ba3fe0a] Adding fifth commit
Date: Wed Jan 17 11:21:16 2024 +0530
1 file changed, 1 insertion(+)
→ git_tutorial_dir git:(main) cat file2.txt
I am creating a file2
I am learning git

```

Here you can see that the content of *file2.txt* changed after the *git cherry-pick* command ran.

Let's see our current directory after giving the *git cherry-pick* command:

```

→ git_tutorial_dir git:(main) ls
file1.txt file2.txt
→ git_tutorial_dir git:(main)

```

If you observe carefully, the sub-directory that I created in the fourth commit is not copied. This is because *git cherry-pick* compares the *diff* of commit to copy with the current *head*

commit and only makes the changes that are not there. Here it saw that *file2.txt* has some lines which were not present in the third commit and copied these. Since the directory was not created in the fifth commit, it did not copy those changes.

Using Git logs for data recovery

Git records all our actions. Therefore, if by mistake we lose work done in a commit, we can always recover that. This happens generally when we delete a branch initially and want that back or when we accidentally hard reset a branch. Here is an example in which we first hard reset a branch and then recover a lost commit.

I am on the 'main' branch and currently it is at 'ba3fe0a6fe2c'.

```

→ git_tutorial_dir git:(main) cat .git/refs/heads/main
ba3fe0a6fe2c3e0469a0c7ee2621b2ec1778fcb5
→ git_tutorial_dir git:(main)

```

Now I do a hard reset for this branch to some previous commit.

```

→ git_tutorial_dir git:(main) git reset --hard
bfb79d72518aebdd5e7df7425bdcefc16e62552
HEAD is now at bfb79d Adding second commit
→ git_tutorial_dir git:(main) cat .git/refs/heads/main
bfb79d72518aebdd5e7df7425bdcefc16e62552
→ git_tutorial_dir git:(main)

```

Effectively, I have now lost all top commits, and have no branch containing those commits. A quick way to find older commits is to use the *git reflog* command. Git records all our actions in the *.git/logs* directory. *git reflog* uses this log folder only to display the history of the repository.

Figure 10 shows the output of *git reflog*. We can clearly see that our *head* was previously at *ba3fe0a* and we have reset the main branch to *bfb79d72*. So now we can create a separate branch with the original commit (*ba3fe0a*).

In case I have lost commit and don't have any reflogs as well to rely on, we can always check for dangling commits (commits that are not part of any branch) using the *git fsck* command.

In this article we have covered Git objects and how they are used by Git to manage different versions of our repository. We also touched upon topics like how Git creates different branches and maintains the *HEAD* pointer, learnt about the internal workings of the *git cherry-pick* command and used Git logs to recover commits that get lost accidentally. Hope this article works as a good starting point for those who want to deepen their understanding of Git. **END** 🐧

By: Anchal Agarwal

The author is a software enthusiast who actively builds and contributes to open source.